Automated Theory Synthesis via Typed Structural Optimization

Halvor Lande[1*]

[1*]Draft affiliation to be supplied, Norway.

Corresponding author(s). E-mail(s): hsl@awc.no;

Abstract

Automated reasoning systems usually optimize a user-supplied theorem or a fixed program specification. This paper studies a different task: selecting which typed theory fragment should be added next to a growing foundational library. Taking Fibonacci-scaled integration debt and schema-based generative calculus as fixed external constraints, we present a large Haskell synthesis engine that evaluates candidate type-theoretic extensions by structural efficiency alone. Candidates are generated as typed abstract syntax trees in a Minimal Base Type Theory grammar, filtered by conservative typing and interface checks, and ranked by minimal positive overshoot above a history-dependent efficiency bar. The scoring path contains no target theorem. In a guided claim-grade lane, the engine recovers a nontrivial 15-step trajectory from the bootstrap of dependent logic through the geometric phase of higher inductive spheres and the Hopf fibration to a temporal-cohesive endpoint. The repository also provides rejected-candidate evidence, ablation runs, and an independent uniform-nu evaluator that confirms the stability of the geometric ordering under a second measurement discipline. The resulting picture is an executable form of automated theory synthesis: symbolic optimization over typed structure rather than proof search for externally given goals.

Keywords: automated reasoning, theory synthesis, type theory, Monte Carlo tree search, program synthesis, homotopy type theory

1 Introduction

1.1 The target-theorem bottleneck

Most successful automated-reasoning systems are driven by a local goal. In theorem proving, the goal is an explicit conjecture or proof state. In theory exploration, the

Table 1 PEN compared with nearby automated-reasoning paradigms.

Family	Input	Search object	Acceptance signal	Output
Goal-conditioned proving	Conjecture or proof state	Proof steps, premises, proof states	Goal solved	Proof or script
Theory exploration	Fixed signature or theory	Equations, lemmas, conjectures	Interestingness or provability inside that theory	Lemma set
Type-directed synthesis	Type or refinement type	Programs inhabiting the target	Type/refinement satisfied	Program term
PEN	Current library state and structural history	Typed library extensions	Bar clearance under $\rho = \nu/\kappa$	Next library entry

user fixes a background theory and the system searches for equations, lemmas, or conjectures internal to that theory. In type-directed synthesis, the user supplies a type or refinement type and the engine searches for a term inhabiting it. This family resemblance covers many of the strongest current systems in automated reasoning, from theory-exploration tools to retrieval-augmented or neural proof search [1–6].

The present paper is about a different optimization problem. The target is not a theorem. The target is the *growth of the library itself*. Starting from a current library state, the engine must decide which typed extension should be admitted next. The search objects are not lemmas but candidate theory fragments: anonymous telescopes describing new type formers, constructors, eliminators, boundary laws, maps, or promoted interface packages. The output is not a proof object but the next accepted library entry and, over time, a trajectory of theory growth.

This reframing matters because it changes both the scoring function and the failure modes. A system can discover many correct lemmas inside a weak theory while still failing to grow the theory in a structurally sustainable direction. Conversely, a candidate extension may be valuable even if it is not immediately tied to a famous human theorem, provided it creates enough future derivational structure to offset its own integration cost. The core question is therefore one of *typed structural optimization*.

1.2 Theoretical inputs as system constraints

This paper is the systems component of a larger PEN program. The heavy metatheoretic inputs are imported rather than reproved. In companion manuscripts, framework extension is shown to incur a coherence-sensitive integration debt whose depth-two form is Fibonacci, and structural novelty is defined by exact schema difference with a tripartite split into generative, cross-interface, and homotopical components [7, 8]. The later temporal-cohesive endpoint is analyzed separately as a companion semantic structure [9]. The present paper treats those results as black-box constraints and asks the computational question: given those constraints, can an executable system recover the predicted trajectory by optimizing typed structure alone?

This division of labor is deliberate. The present contribution is not a new proof of Fibonacci scaling. It is the implementation of a large Haskell engine that computes

the corresponding objective over typed ASTs, manages the combinatorial search space, and records positive and negative evidence about what survives the bar.

1.3 The core thesis

The central claim of the paper is the following.

> *A typed structural optimization engine with no target theorem in its scoring path recovers a nontrivial geometric trajectory, computationally verifying the theoretical predictions and demonstrating a new paradigm for automated mathematical discovery.*

The phrase "no target theorem in its scoring path" is the crucial boundary condition. It does not mean the whole system is free of guidance. It means that the *score* assigned to a candidate is computed from structural quantities only: novelty ν, irreducible effort κ, and the history-sensitive bar induced by integration debt. Human mathematical taste does not appear in the evaluator. Search-control heuristics still matter, and we discuss them explicitly in Section 8.

1.4 Scope and methodology

The paper is written for a JAR audience and therefore emphasizes what the engine computes, how the search surface is controlled, and where the artifact evidence is strongest. The main repository contains two mechanized pillars: a Cubical Agda proof layer and a Haskell synthesis and verification engine [10]. The Haskell side generates candidates, checks them conservatively, computes novelty and effort, applies bar-based selection, and emits replay telemetry. The Agda side mechanizes recurrence facts, abstraction barriers, and bridge contracts. Our methodology is to treat the codebase itself as the primary systems source and to document the strongest artifact-supported claims while keeping conjectural claims clearly separated.

2 Formal optimization over typed ASTs

2.1 The state and action space

Definition 2.1 (Library state)**.** *A library state at time n is a pair*

$$S_n = (\mathcal{B}_n, H_n),$$

where $\mathcal{B}_n$ is the current sequence of sealed library entries and $H_n = ((\nu_1, \kappa_1), \ldots, (\nu_n, \kappa_n))$ is the realized score history used to compute the next bar. In the implementation, $\mathcal{B}_n$ is represented by a list of `LibraryEntry` *values carrying constructors, path dimensions, and capability flags required by subsequent search.*

Definition 2.2 (Action space)**.** *An action at state S_n is a typed candidate extension T written as an anonymous* MBTT *telescope over the active library. The admissible action set $A(S_n)$ consists of those telescopes that pass the conservative telescope checker, remain connected to the current library, and satisfy the current coherence-window reference discipline.*

The important implementation fact is that the search surface is not the untyped space of arbitrary bitstrings. The basic search objects are typed ASTs in a restricted grammar. The module `MBTTEnum.hs` enumerates candidates by budget-splitting over this grammar, while `TelescopeCheck.hs` rejects malformed telescopes, disconnected candidates, and obvious definitional junk. Each candidate carries multiple cost views, including bit cost, clause count, and AST node count.

2.2 The PEN algorithm as an executable objective

The evaluator follows the companion mathematics closely enough that it is useful to state the objective formally.

Definition 2.3 (Dual costs)**.** *For a candidate T over library $\mathcal{B}$:*

(i) $\kappa(T \mid \mathcal{B})$ is its irreducible construction effort, computed in the default lane by desugared clause count and exposed additionally in entry-count and bit-cost variants.

(ii) $\nu(T \mid \mathcal{B})$ is its structural novelty, computed either from the AST rule extractor (structural lane) or from the independent uniform evaluator (verification lane).

The efficiency of T is

$$\rho(T \mid \mathcal{B}) = \frac{\nu(T \mid \mathcal{B})}{\kappa(T \mid \mathcal{B})}.$$

Definition 2.4 (History-sensitive bar)**.** *Let Δ_n be the integration latency at step n, computed from the chosen coherence-window recurrence. Define*

$$\Phi_n = \frac{\Delta_n}{\Delta_{n-1}}, \qquad \Omega_{n-1} = \frac{\sum_{i=1}^{n-1} \nu_i}{\sum_{i=1}^{n-1} \kappa_i}, \qquad \mathrm{Bar}_n = \Phi_n \Omega_{n-1}.$$

In the default window $d = 2$, the recurrence is Fibonacci-based via the executable function `dBonacciDelta`.

Definition 2.5 (Executable PEN objective)**.** *Given a state $S_n = (\mathcal{B}_n, H_n)$, the engine selects*

$$T^* \in \operatorname{argmin}\Big\{\rho(T \mid \mathcal{B}_n) - \mathrm{Bar}_{n+1} \;\Big|\; T \in A(S_n),\ \rho(T \mid \mathcal{B}_n) \geq \mathrm{Bar}_{n+1}\Big\},$$

with deterministic tie-breaks by smaller κ, then stronger interface density, then a stable canonical key.

Minimal positive overshoot, rather than raw maximization of ρ, is a substantive part of the objective. The engine is not asked to find the candidate with the largest instantaneous novelty-to-effort ratio. It is asked to find the least supercritical candidate that clears the current bar.

Proposition 2.6 (Target-free scoring)**.** *The score assigned to a candidate in the structural and strict claim-grade lanes depends only on the candidate telescope, the current library state, and realized history. It contains no parameter representing a target theorem, proof state, conjecture, or external utility label.*

Proof. In the current codebase, the evaluation path factors through the telescope checker, the structural or uniform novelty evaluator, the selected κ mode, and the bar

Algorithm 1 Executable PEN selection loop

Require: library state $S_n = (\mathcal{B}_n, H_n)$, enumeration config E, optional MCTS config M

$C \leftarrow \textsc{EnumerateMBTT}(\mathcal{B}_n, E)$
$C \leftarrow \textsc{CheckAndPrune}(C, \mathcal{B}_n)$
$C \leftarrow \textsc{CanonicalDeduplicate}(C)$
for all $T \in C$ **do**
    compute $\kappa(T)$, $\nu_G(T)$, $\nu_C(T)$, $\nu_H(T)$, $\nu(T)$, and $\rho(T)$
end for
compute Bar_{n+1} from H_n and the active window depth
$V \leftarrow \{T \in C : \rho(T) \geq \mathrm{Bar}_{n+1}\}$
if $V = \varnothing$ and M is enabled **then**
    $V \leftarrow \mathrm{MCTS}\textsc{Fallback}(\mathcal{B}_n, H_n, M)$
end if
return the least-supercritical candidate in V under deterministic tie-breaks

function computed from prior realized (ν, κ) pairs and the chosen window depth. The claim-grade source guards in `RunAbInitio.hs` reject `REF` and `PAPER` sources for strict and structural discovery. Search guidance may still narrow the candidate frontier, but the acceptance score itself is structural. □

2.3 Target-free scoring versus guided search

Because the phrase "target-free" is easy to overread, we make the distinction explicit already here. The current engine is target-free at the level of *evaluation*. It is not entirely unguided at the level of *search control*. The claim-grade lane still uses agenda mechanisms, bounded frontiers, canonical quotienting, and late-phase ranking controls. This distinction is not a weakness of the paper; it is part of the scientific result. The system is already strong enough to separate a structurally defined reward from a pragmatically guided traversal of the candidate manifold.

3 System architecture: the Haskell engine

3.1 Engine design and scale

The current repository is an executable research artifact rather than a single-purpose script. The repository README describes two primary pillars: a proof pillar in Cubical Agda and a computation pillar in Haskell [10]. The architecture document summarizes the same split at maintainer level. The JAR draft already present in the codebase reports roughly 50 Haskell modules and about 19.5k lines under `engine/src`, together with a parallel Agda mechanization, scripts, and run artifacts; that is the scale at which the present manuscript should be read.

Table 2 Main architectural groups in the current codebase.

Group	Representative modules	Role
Search-space generation	`MBTTEnum`, `AgendaSearch`, `Generator`, `MCTS`	Typed candidate generation under budgets; fallback or portfolio exploration when enumeration is insufficient.
Checking and canonicalization	`TelescopeCheck`, `MBTTCanonical`, `Telescope`	Conservative well-formedness checks, connectedness tests, canonical keys, telescope utilities, and structural classification.
Scoring and selection	`StructuralNu`, `UniformNu`, `TelescopeEval`, `RunAbInitio`	Computation of ν, κ, ρ, the bar, and final deterministic selection.
Search guidance	`ProofState`, `ProofRank`, `CoherenceWindow`	Goal profiles, debt graphs, capability signatures, premise buckets, and coherence-derived budgets.
Reporting and bridge	`MBTTDecode`, `AgdaExport`	Post-hoc semantic labels, bridge payloads, Agda stubs, and non-interfering reporting channels.
Validation	`AcceptanceSuite` and replay scripts	Golden-sequence tests, invariants, regression checks, and benchmark harnesses.

3.2 MBTT-first candidate generation

The most important architectural evolution relative to early PEN manuscripts is the move toward MBTT-first search. The module `MBTTEnum.hs` implements exhaustive budget-split enumeration over the typed MBTT grammar rather than relying exclusively on template-first generation. Its default configuration is explicit in the code:

Parameter	Default value
bit budget per candidate	40
maximum telescope entries	8
maximum AST depth	5
maximum returned candidates	5000

Each `MBTTCandidate` carries four relevant pieces of metadata: the anonymous telescope itself, bit cost, desugared clause count, and total AST node count. This is the implementation bridge between formal novelty and ordinary systems telemetry.

The enumeration itself is typed and structural. Pi-types are generated by budget splitting over domain and codomain; identity forms split budget over three arguments; library gating reuses the current library interface; and the resulting telescopes are filtered by the conservative checker, connectedness, and the active reference window. This is already far from brute-force term enumeration in an untyped grammar.

3.3 Type checking, normalization, and canonical quotienting

`TelescopeCheck.hs` is conservative rather than complete. It aims to reject obviously malformed telescopes without pretending to solve full equivalence or full dependent type checking for the entire search surface. The checks enforce library-reference bounds, variable-scope discipline, binder sanity, and certain term-form restrictions. A candidate must also be structurally connected to the recent library and respect the active window references.

Canonicalization is similarly explicit about its limits. The current quotient is deterministic and intentionally modest: recursive structural normalization, hashed canonical encodings, and stable canonical keys. It does not attempt alpha-normalization,

beta-eta equivalence, or symmetry quotients. This is a sensible engineering compromise for reproducibility. The phase-2 differential report cited in the repository records a 48.33% candidate reduction on a bounded differential gate after canonical quotienting, which is strong enough to matter operationally while remaining simple enough to audit.

4 The search controller: MCTS

4.1 Why a search controller is needed

Even a typed AST manifold becomes combinatorially heavy as the library grows. The reason is not mysterious: the admissible surface expands with the active interface, candidate well-formedness is history sensitive, and the reward is computed only after conservative checking, canonicalization, and novelty evaluation. Exhaustive enumeration is therefore strongest in early and mid phases, whereas later phases motivate a portfolio or fallback controller.

The codebase addresses this with a Monte Carlo Tree Search layer. Conceptually, MCTS explores partial telescopes as states of a finite-horizon Markov decision process. Selection uses UCT-style optimism, expansion adds structurally valid next actions, rollouts estimate the downstream efficiency ratio, and backpropagation updates utility estimates [11, 12]. In PEN, however, MCTS is not the main scientific novelty; it is a scaling mechanism attached to a structural evaluator.

4.2 The implemented MCTS layer

The module `MCTS.hs` exposes the controller with transparent defaults:

Parameter	Default value
iterations	10,000
maximum candidate κ	5
maximum AST depth	2
UCT exploration constant	1.414
reported top-K candidates	10
progressive widening coefficient	1.0
progressive widening exponent	0.5
random seed	42

A node in the MCTS tree is a partial telescope. The action set consists of valid structural extensions produced by `validActions`. Reward is tied to efficiency on completed telescopes, with viability measured against the current bar and deterministic post-checking before insertion into the candidate pool. In the current `RunAbInitio.hs` code path, MCTS serves as a fallback or portfolio lane when the expected relevant κ exceeds the enumeration horizon or when enumeration yields no viable candidate.

An important empirical point is that the currently documented full strict replay used for telemetry does *not* exercise MCTS. That is valuable rather than embarrassing. It shows that the available 15-step recovery can already be obtained by typed enumeration, agenda narrowing, and structural scoring. The MCTS layer is therefore part of the intended scaling story, but not necessary for the strongest present replay claim.

Algorithm 2 MCTS fallback over partial telescopes

Require: current library $\mathcal{B}_n$, MCTS configuration M

initialize root at the empty or seeded partial telescope
for $i = 1$ to `mctsIterations` **do**
  select a node using UCT with progressive widening
  expand by one structurally valid action
  simulate a bounded rollout using valid telescope actions
  evaluate completed candidates by the same ν/κ and bar criteria as in Algorithm 1
  backpropagate rollout utility to the selected path
end for
return top-ranked viable completions for deterministic post-selection

5 Experimental results: recovering the trajectory

5.1 The guided claim-grade lane

The main experimental lane is deliberately narrower than the full conceptual search space. The codebase distinguishes between benchmark-style replay modes and claim-grade discovery modes. In the latter, the evaluator rejects `REF` and `PAPER` sources, computes ν and κ from structural information only, and builds the bar from discovered history. The source guards are visible in `RunAbInitio.hs`; names are attached post hoc by the reporting decoder and are kept non-interfering.

A minimal reproduction path, directly supported by the repository README and pseudo-code notes, is:

```
cd engine
cabal build all
cabal run ab-initio -- --strict --max-steps 15 --skip-validation --skip-mcts
cabal run acceptance
```

The same repository also supports benchmark harnesses and shorter shadow-prefix runs, for example through `./scripts/benchmark.sh quickstart_benchmark` and the `--phase1-shadow` preset. The point of the claim-grade lane is not that it searches everything. The point is that within a constrained but auditable lane, the engine selects and rejects candidates by structural score rather than by theorem targeting.

5.2 The exact step-5 audit as a worked example

The first genuinely geometric acceptance event is step 5. At this point the active library contains exactly the bootstrap deductive core recovered in steps 1–4: Universe, Unit, Witness, and Π/Σ-types. In the baseline structural lane the bar is then $\mathrm{Bar}_5 = 2.14$, and the circle candidate satisfies

$$\kappa(S^1) = 3, \qquad \nu(S^1) = 7, \qquad \rho(S^1) = \frac{7}{3} \approx 2.33.$$

The imported generative-calculus account explains the homotopical part of this score: for the circle, $m = 1$ boundary reduction and maximal path dimension $d = 1$, so the topological projection theorem gives $\nu_H = m + d^2 = 2$. The remaining five schema families arise from formation, introduction, and the dependent elimination or mixed-interface consequences unlocked by adding the first non-trivial path object.

This makes step 5 a useful systems-level sanity check because it already separates geometric from discrete candidates. Bare naturals have $\rho \leq 4/3$, classical excluded middle has $\rho \leq 1$, and power-set style axioms also remain near 1 in the companion audits. All of them fall below the same bar that the circle clears. The engine therefore does not need a semantic preference for topology to choose S^1 at step 5; the choice is already forced by the structural objective imported from the companion theory.

5.3 Computational recovery of the 15 steps

Table 3 records the main structural-discovery trajectory from the repository's baseline structural $d = 2$ artifact. It recovers the full sequence from the empty library to the temporal-cohesive endpoint.

The resulting totals are $\sum \nu_i = 359$ and $\sum \kappa_i = 64$ in the structural lane. Two details are particularly important for a systems audience. First, the trajectory is not recovered by maximizing raw ρ. At step 4, Π/Σ with $\rho = 1.6667$ is preferred to a competing candidate at $\rho = 1.75$ because Pi is the smaller positive overshoot above the bar 1.50. Likewise, step 14 prefers Hilbert with $\rho = 6.8889$ to a candidate around 7.11 because the former is closer to the bar. Second, the recovered labels are reporting names attached after structural selection; they are not part of the score path.

5.4 Telemetry, runtime envelope, and AST evaluations

The repository's strongest telemetry is in the strict MBTT-first replay traces. These record raw frontier sizes, canonical survivors, viable candidates, and source provenance. They also expose memory guards and shorter CI-oriented runtime gates.

The engineering message of Table 5 is that the recovered path is narrow. Later steps do not arise from scanning a huge undifferentiated frontier; they arise after typing, agenda pruning, and quotienting have already compressed the candidate surface. This is also why the current artifact trail should be described as a *guided claim-grade lane* rather than as a theorem about exhaustive unguided search.

6 Empirical validation

6.1 Rejected candidates analysis

Positive recovery is only half of the evidence. The other half is the systematic discard pile. The repository and companion manuscripts both emphasize that representative discrete and classical packages are generated and then rejected for structural reasons. Table 6 gives a compact summary.

Two kinds of negative evidence are worth distinguishing. Domain-level rejections show that many mathematically respectable packages simply do not clear the bar. Policy ablations show that changing the selector destroys the trajectory. In the repository,

disabling canonical priority or switching from minimal overshoot to pure max-ρ leads to immediate degeneration toward unnamed fragments or one-clause artifacts. The sequence is therefore not robust to arbitrary objective choices; it depends on the specific PEN objective.

6.2 Independent confirmation via the uniform-ν algorithm

The repository contains a second evaluator, implemented in `UniformNu.hs`, whose job is not to drive the main selector but to test whether the ordering survives a distinct measurement discipline. The uniform evaluator computes novelty in three stages:

$$\nu_h^{\text{unif}}(X \mid \mathcal{B}) = s_h(X \mid \mathcal{B}) + f(X \mid \mathcal{B}) + a(X \mid \mathcal{B}), \tag{1}$$

where s_h is the number of distinct non-trivial schema families induced by newly inhabited types up to depth h, f counts genuinely new type formers, and a adds adjoint-completion credit for eliminative consequences exposed by the candidate. The module-level record `UniformNuResult` stores the raw new-type count, schema count, per-depth profile, and adjoint credit explicitly.

Remark 6.1 (Role of the uniform evaluator)**.** *In structural discovery mode, `UniformNu` is not on the scoring path. It is used as a verification lane. This design choice is important because it makes agreement across lanes scientifically meaningful: the structural selector is not being retrofitted to match the uniform audit.*

6.3 Verifying ordering constraints

The strongest stable empirical conclusion is not that every late-step absolute novelty value is frozen. It is that the *order* of the trajectory is stable across lanes more often than the magnitudes are. The current repository contains several such lanes.

The honest reading of Table 7 is the following. Order stability is the main success. Late-step absolute magnitudes still drift across replay harnesses and should be treated as engineering debt, not swept away by rhetoric. For a JAR paper this is a virtue: it shows exactly where the executable evidence is strongest and where the measurement stack still needs consolidation.

7 Related work and systems comparison

7.1 Conjecture generation and theory exploration

QuickSpec, Hipster, and IsaCoSy are the closest neighboring systems conceptually because they also operate without a single user-supplied theorem at the front of the pipeline [1–3]. The key difference is that their background theory is fixed. They explore facts inside a user-provided signature. PEN instead changes the signature itself. Its search objects are theory fragments, not equations. The distinction is not cosmetic: it changes the objective from "find latent equalities" to "choose the next library extension under rising integration debt."

Historical automated theory-formation systems such as AM and Colton's work also matter as intellectual background [13, 14]. What PEN adds is a typed structural

objective tied to an explicit bar dynamics and a concrete mechanized connection to type-theoretic library growth.

7.2 Type-driven synthesis

Systems such as Synquid solve a sharply posed synthesis task: given a target type or refinement type, construct a term satisfying it [4]. That is a powerful and successful paradigm, but it differs from the present task in two ways. First, the target remains external to the synthesized artifact. Second, success is local to the requested specification. PEN instead evaluates the impact of a candidate on the *future topology of the library*. The system is therefore not a competitor to refinement-type synthesis so much as a different level of optimization.

7.3 Neural theorem proving and proof-guided learning

Modern neural or language-model-based reasoning systems have made rapid progress on premise selection, tactic generation, and formal proof search [5, 6]. They remain overwhelmingly target directed: the system is trained or prompted to prove a conjecture, finish a proof state, or solve a specific benchmark family. PEN is different in two directions. It is symbolic rather than learned in its main score path, and its objective is theory growth rather than proof completion. This is precisely why the phrase "no target theorem in its scoring path" matters.

8 Discussion: autonomy versus guidance

8.1 A clear-eyed boundary analysis

The right way to describe the system's agency is neither dismissive nor grandiose.

What is already autonomous is substantial. The engine combines typed AST fragments, filters them by conservative structural constraints, computes novelty and effort exactly in the active lane, evaluates bar clearance, rejects large families of classical or discrete candidates, and chooses winners by deterministic minimal overshoot. The recovered ordering, the discard pile, and the post-hoc reporting tables are all downstream of that machinery.

What remains guided is also substantial. The search manifold is deliberately restricted by the MBTT grammar seed, bounded frontiers, agenda search, canonical quotienting, and late-phase ranking controls. MCTS exists as a scaling controller, but the strongest currently documented full replay does not depend on it. The correct claim is therefore not "fully autonomous mathematics." It is "an executable symbolic engine whose evaluation is target free and whose present search is guided enough to be tractable and auditable."

8.2 Scalability limits

Several engineering limits should remain prominent in any serious JAR version.

First, canonicalization is intentionally weak. The current quotient collapses only deterministic structural duplicates; it does not solve alpha, beta, eta, or symmetry

equivalence. Second, late-step novelty magnitudes drift across replay lanes. Third, the repository still lacks a direct head-to-head executable benchmark against tools such as QuickSpec or Hipster on a controlled overlapping domain. Fourth, claims beyond step 15 would currently require stronger pruning, broader MCTS use, or both. For these reasons the paper should avoid vague claims of broad scalability and instead emphasize the existing artifact contracts: full 15-step replays, shadow-prefix CI gates, rejection evidence, and explicit barriers against name leakage into the evaluator.

9 Conclusion

This paper has presented PEN as a system for automated theory synthesis rather than theorem proving. The search objects are typed library extensions; the score is structural efficiency under a coherence-sensitive bar; and the recovered output is a trajectory of theory growth. Within a guided claim-grade lane, the engine recovers a highly nontrivial 15-step sequence from the bootstrap of dependent logic through the geometric core of homotopy theory to a temporal-cohesive endpoint. It simultaneously rejects large families of discrete or classical alternatives for the same structural reasons predicted by the companion mathematics.

The strongest contribution is conceptual and executable at once: it is possible to build an automated-reasoning system whose scoring path contains no target theorem and yet which discovers mathematically recognizable structure by optimizing typed theory fragments. That does not settle the broader problem of autonomous mathematical discovery, but it opens a new and concrete paradigm for it.

Table 3 Structural-discovery trajectory in the baseline $d = 2$ lane.

Step	Structure	ν	κ	ρ	Bar	Δ_n
1	Universe	1	2	0.50	0.50	1
2	Unit	1	1	1.00	0.50	1
3	Witness	2	1	2.00	1.33	2
4	Π/Σ	5	3	1.67	1.50	3
5	S^1	7	3	2.33	2.14	5
6	Trunc	8	3	2.67	2.56	8
7	S^2	10	3	3.33	3.00	13
8	S^3	18	5	3.60	3.43	21
9	Hopf	17	4	4.25	4.01	34
10	Cohesion	19	4	4.75	4.46	55
11	Connections	26	5	5.20	4.91	89
12	Curvature	34	6	5.67	5.43	144
13	Metric	46	7	6.57	5.99	233
14	Hilbert	62	9	6.89	6.68	377
15	DCT	103	8	12.88	7.40	610

Table 4 Resource and search telemetry reported by the current artifact trail.

Signal	Value	Comment
Raw candidate ASTs on the winning strict replay path	123	Total number of raw candidate telescopes evaluated across the 15 accepted steps.
Canonical/viable candidate ASTs	77	Post-quotient survivors; average dedupe ratio reported as 0.8184 in the existing draft.
Peak memory in an early strict replay log	around 1 GiB	Reported in the current JAR-draft artifact notes.
CI shadow-prefix wall-clock envelope	30 seconds through step 7	Provisioned-runner target noted in the artifact documentation.
Adaptive search budgets	enabled	Enumeration and MCTS budgets are memory aware in the current controller.

Table 5 Per-step frontier telemetry from a full strict MBTT-first replay.

Step	Structure	Raw	Canonical	Viable	Source
1	Universe	12	11	11	ENUM_MBTT
2	Unit	11	11	11	ENUM_MBTT
3	Witness	7	3	3	ENUM_MBTT
4	Pi	32	22	22	AGENDA
5	S^1	4	2	2	AGENDA
6	Trunc	37	9	9	AGENDA
7	S^2	5	5	5	AGENDA
8	S^3	2	2	2	AGENDA
9	Hopf	2	1	1	AGENDA
10	Cohesion	1	1	1	AGENDA
11	Connections	2	2	2	AGENDA
12	Curvature	2	2	2	AGENDA
13	Metric	1	1	1	AGENDA
14	Hilbert	3	3	3	AGENDA
15	DCT	2	2	2	AGENDA

Table 6 Representative rejected candidates in the current PEN artifact narrative.

Candidate	ν	κ	ρ	Reason for rejection
Natural numbers $\mathbb{N}$	≤ 4	3	≤ 1.33	Too little cross-interface growth once higher-path candidates are admissible.
Classical LEM	≤ 1	1	≤ 1.00	Minimal novelty, no higher-path or modal amplification.
Power-set style axiom	≤ 2	2	≤ 1.00	Weak interaction per clause relative to the dynamic bar.
Ordinal arithmetic	≤ 6	5	≤ 1.20	Discrete novelty remains subcritical.
Measure theory	≈ 12	6	≈ 2.0	Arrives too late with insufficient efficiency.
Elementary topos axioms	≈ 15	8	≈ 1.9	High cost, modest local amplification in this objective.
Symplectic geometry	≈ 25	5	≈ 5.0	Closest geometric near miss; still below the relevant late bar.

Table 7 Replay and verification lanes visible in the current repository state.

Lane	15-step order	Total ν	Comment
Paper-calibrated replay	Yes	356	Legacy comparison lane; late steps use $(43, 60, 105)$ for Metric/Hilbert/DCT.
Structural discovery ($d = 2$)	Yes	359	Primary publication-grade structural baseline; late steps use $(46, 62, 103)$.
Strict MBTT-first replay	Yes	artifact drift	Current notes report the same order, with late-step strict magnitudes varying across artifacts (for example DCT recorded as 77 in one snapshot and 88 in a later strict note).
Window sweep $d = 1$	Yes in benchmark artifact	347	Inferior to $d = 2$ under benchmark total novelty.
Window sweep $d = 3$	Yes in benchmark artifact	195	Much weaker than $d = 2$ under the same benchmark metric.
Uniform-ν audit	Geometric phase order preserved	not directly comparable	Independent verification lane based on inhabited-type density plus adjoint-completion credits.

Availability of artifacts

The mechanization, Haskell engine, replay scripts, and companion manuscripts are available in the public PEN repository [10]. The repository README also documents basic build commands, environment requirements, and the split between the Agda proof pillar and the Haskell computation pillar.

References

[1] Claessen, K., Smallbone, N., Hughes, J.: Quickspec: Guessing formal specifications using testing. In: Tests and Proofs (TAP), pp. 6–21. Springer, Berlin, Heidelberg (2010)

[2] Johansson, M., Dixon, L., Bundy, A.: Conjecture synthesis for inductive theories. Journal of Automated Reasoning **47**(3), 251–289 (2011) https://doi.org/10.1007/s10817-010-9193-y

[3] Johansson, M., Rosén, D., Smallbone, N., Claessen, K.: Hipster: Integrating theory exploration in a proof assistant. In: Intelligent Computer Mathematics (CICM), pp. 108–122. Springer, Berlin, Heidelberg (2014)

[4] Polikarpova, N., Kuraj, I., Solar-Lezama, A.: Program synthesis from polymorphic refinement types. In: Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), pp. 522–538 (2016). https://doi.org/10.1145/2908080.2908093

[5] Yang, K., Swope, A.M., Gu, A., Chalamala, R., Song, P., Yu, S., Godil, S., Prenger, R.J., Anandkumar, A.: Leandojo: Theorem proving with retrieval-augmented language models. In: Advances in Neural Information Processing Systems 36 (NeurIPS 2023) (2023)

[6] Trinh, T.H., Wu, Y., Le, Q.V., He, H., Luong, T.: Solving olympiad geometry without human demonstrations. Nature **625**(7995), 476–482 (2024) https://doi.org/10.1038/s41586-023-06747-5

[7] Lande, H.: Coherence Depth and Fibonacci Scaling in Intensional Type Theory. Companion manuscript in the PEN mechanization repository. Draft manuscript, 2026 (2026)

[8] Lande, H.: A Generative Calculus for Type-Theoretic Complexity and the Geometric Core. Companion manuscript in the PEN mechanization repository. Draft manuscript, 2026 (2026)

[9] Lande, H.: The Differential Completion Theorem: Temporal Cohesion and Local Algorithmic Halting in Type-Theoretic Framework Growth. Companion manuscript in the PEN mechanization repository. Draft manuscript, 2026 (2026)

[10] Lande, H.: PEN Mechanization Repository. GitHub repository. https://github.com/efficient-novelty/mechanization. Accessed March 2026 (2026)

[11] Kocsis, L., Szepesvári, C.: Bandit based monte-carlo planning. In: Proceedings of the 17th European Conference on Machine Learning (ECML), pp. 282–293. Springer, Berlin, Heidelberg (2006)

[12] Browne, C.B., Powley, E., Whitehouse, D., Lucas, S.M., Cowling, P.I., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S., Colton, S.: A survey of monte carlo tree search methods. IEEE Transactions on Computational Intelligence and AI in Games **4**(1), 1–43 (2012) https://doi.org/10.1109/TCIAIG.2012.2186810

[13] Lenat, D.B.: Automated theory formation in mathematics. In: Proceedings of the 5th International Joint Conference on Artificial Intelligence (IJCAI), pp. 833–842 (1977)

[14] Colton, S.: An application-based comparison of automated theory formation and inductive logic programming. Electronic Transactions on Artificial Intelligence **4**, 97–117 (2000)